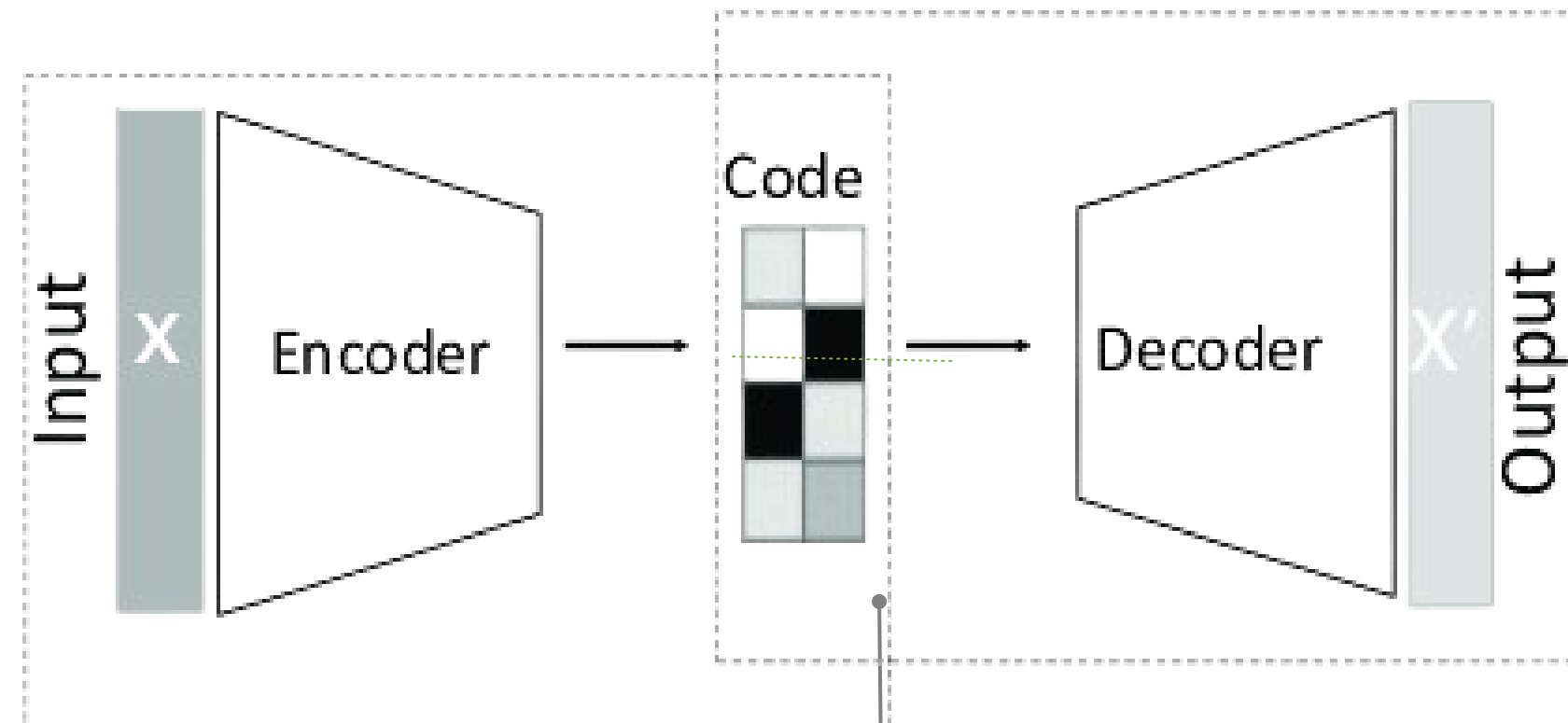


Auto-encodeurs

Compréhension et Applications



Supervisé par : Mohamed Charradi
Réalisé par : Siham Kalach

Plan du Chapitre 09

Introduction

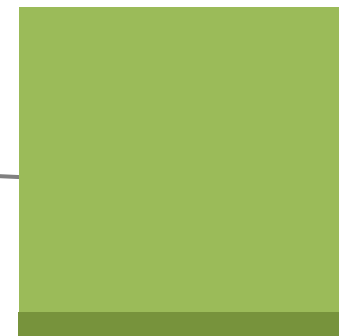
Qu'est-ce qu'un auto-encodeur ?



Auto-Encoder Vs PCA

Différences entre Auto-encodeur et PCA

- Méthode de Transformation
- Nature des Relations
- Apprentissage et Optimisation



Architecture d'un auto-encodeur

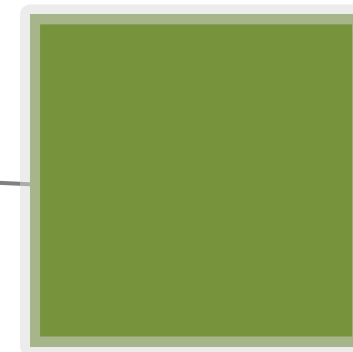
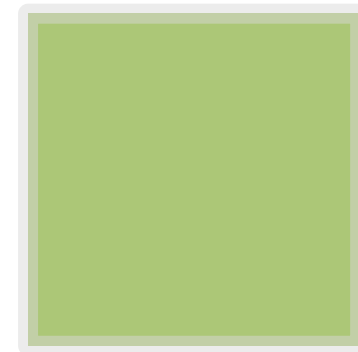
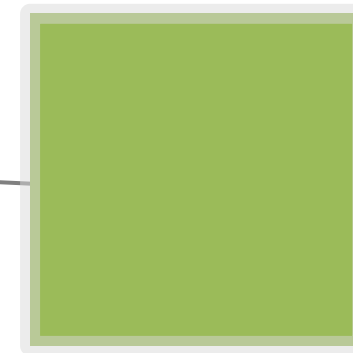
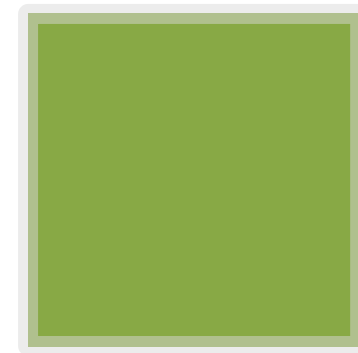
1. Encodeur (Encoder)
2. Bottleneck (ou espace latent)
3. Décodeur (Decoder)

Étapes de Conception

- Construire un simple auto-encodeur
- Le but de auto-encodeur

Comprendre les Autoencodeurs : Les notions essentielles

Ajustement des Hyperparamètres d'un Autoencodeur



Types d'auto-encodeurs

1. Denoising Autoencoder
2. Sparse Autoencoder
3. Variational Autoencoder
4. Convolutional Autoencoder

Exemple pratique avec Keras

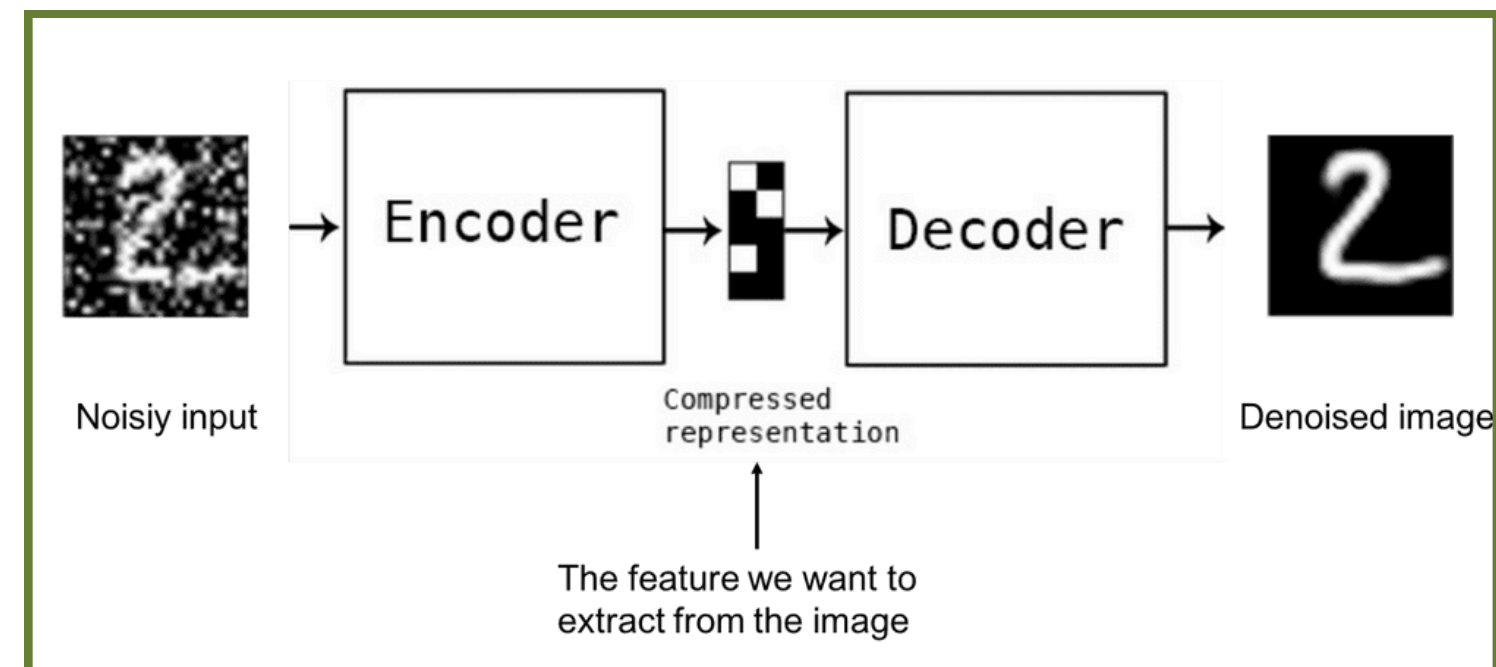
Applications des auto-encodeurs

Limitations des auto-encodeurs

Introduction

Qu'est-ce qu'un Auto-encodeur ?

- Les auto-encodeurs sont un type de réseau de neurones **non supervisé** utilisé principalement pour la réduction de dimension, la compression, la reconstruction, ou encore la détection d'anomalies.
- Ils apprennent à **encoder les données** en une représentation compacte, puis à **les décoder** pour retrouver les données d'origine.

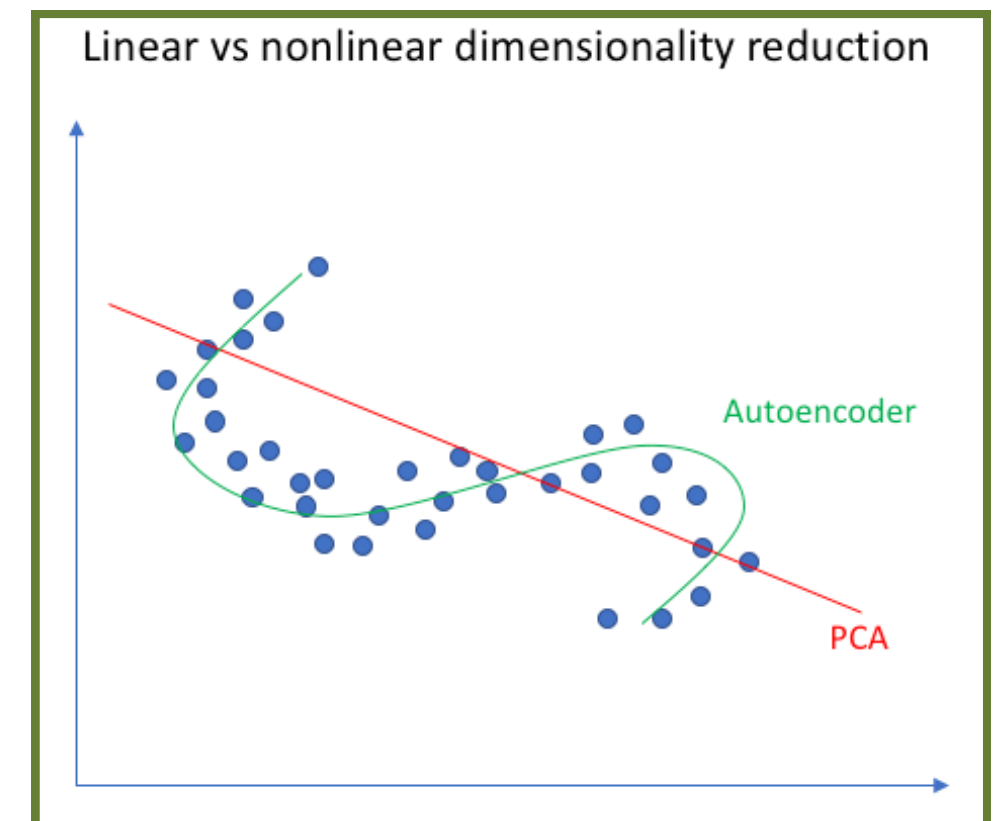


Différences entre Auto-encodeur et PCA

Bien que l'auto-encodeur et l'Analyse en Composantes Principales (PCA) servent à des fins similaires, à savoir la réduction de dimensionnalité, ils diffèrent de manière significative dans leur fonctionnement :

1. Méthode de Transformation :

- **PCA** : Utilise une méthode linéaire basée sur les composantes principales. Elle transforme les données en de nouvelles dimensions ordonnées selon leur variance, en maximisant la variance des données projetées.
- **Auto-encodeur** : Utilise un réseau de neurones pour apprendre une transformation non linéaire des données. Cela permet à l'auto-encodeur de capturer des relations complexes dans les données, qui ne peuvent pas être saisies par PCA.

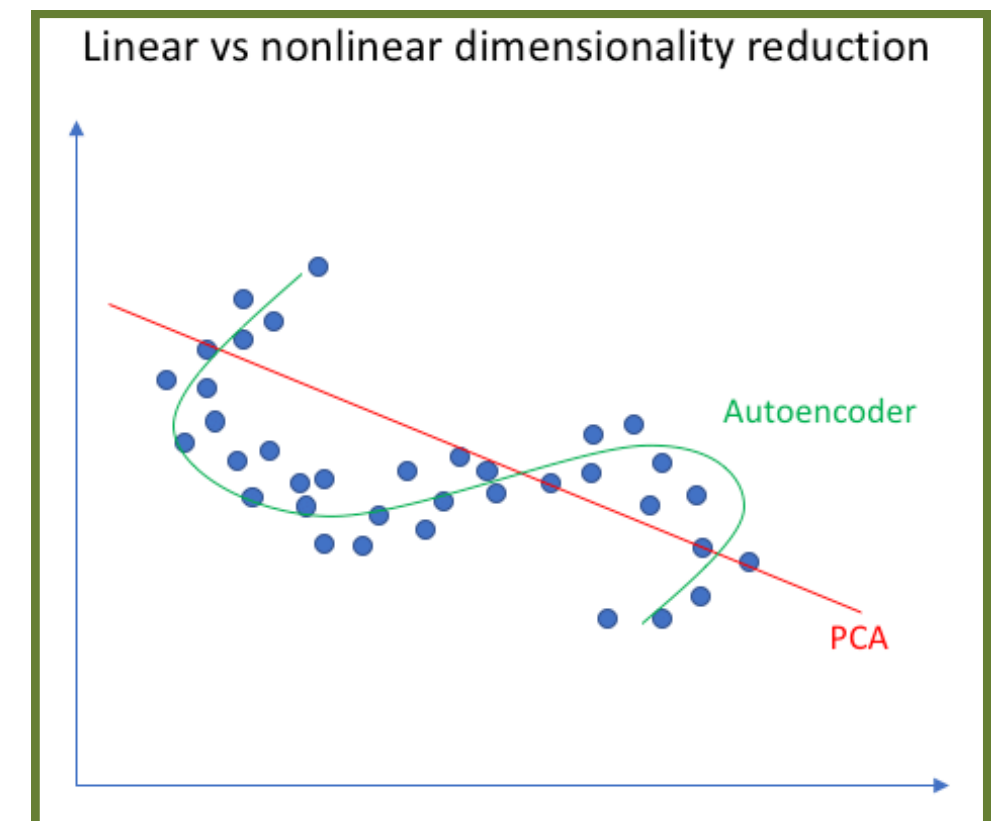


Différences entre Auto-encodeur et PCA

Bien que l'auto-encodeur et l'Analyse en Composantes Principales (PCA) servent à des fins similaires, à savoir la réduction de dimensionnalité, ils diffèrent de manière significative dans leur fonctionnement :

2. Nature des Relations :

- **PCA** : Repose sur des relations linéaires entre les variables. Elle est efficace lorsque les données suivent une structure linéaire.
- **Auto-encodeur** : Peut apprendre des relations non linéaires entre les données. Cela en fait un modèle plus puissant pour des données complexes, comme des images .

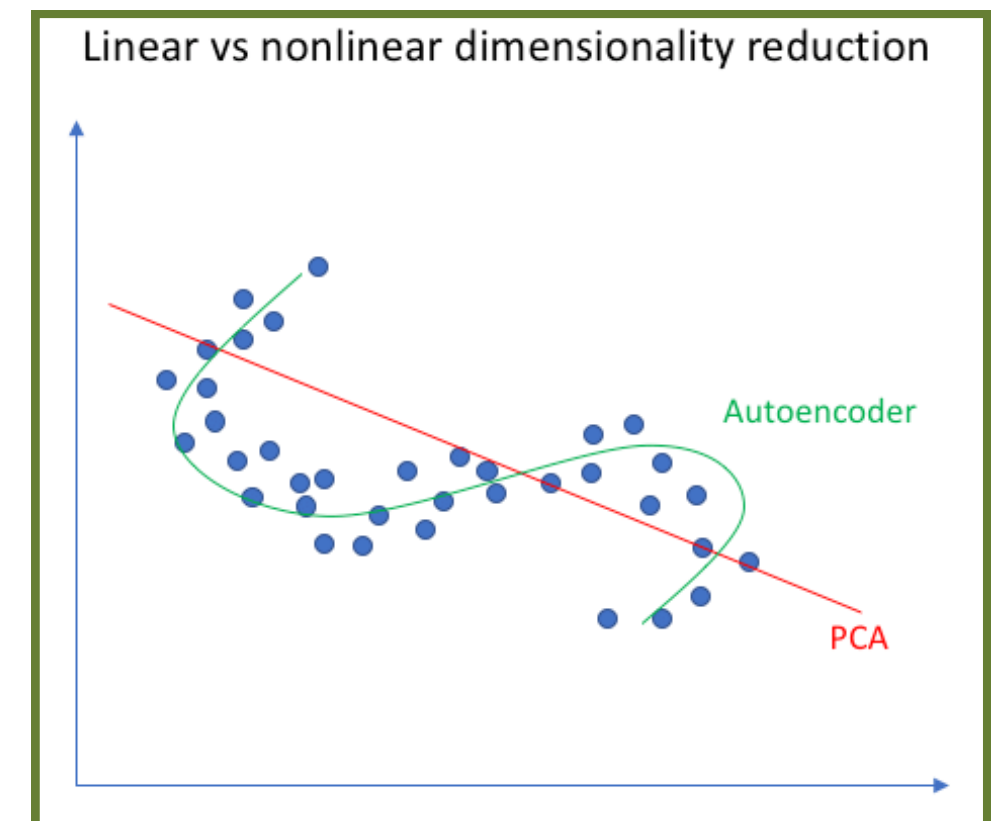


Différences entre Auto-encodeur et PCA

Bien que l'auto-encodeur et l'Analyse en Composantes Principales (PCA) servent à des fins similaires, à savoir la réduction de dimensionnalité, ils diffèrent de manière significative dans leur fonctionnement :

3.Apprentissage et Optimisation :

- **PCA** : Ne nécessite pas de processus d'apprentissage ni de paramétrage. C'est une méthode déterministe qui calcule directement les directions principales à partir des données.
- **Auto-encodeur** : Nécessite un processus d'entraînement avec des paramètres ajustables. Les poids du réseau de neurones sont optimisés pour minimiser l'erreur de reconstruction entre l'entrée et la sortie.

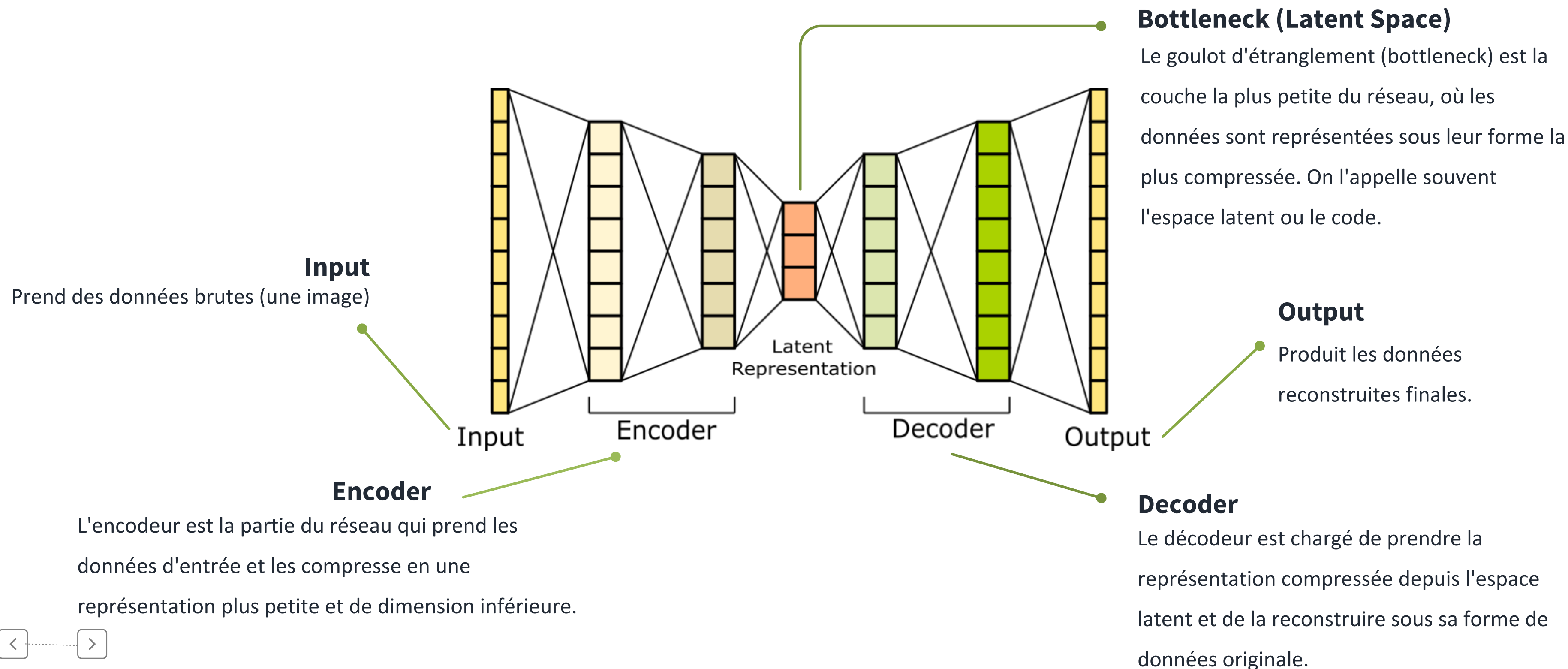


Architecture d'un Auto-encodeur



Architecture d'un Auto-encodeur

Comment un Autoencodeur est Conçu ?



Architecture d'un Auto-encodeur

Comment un Autoencodeur est Conçu ?

Il est constitué de trois parties principales :

1.Encodeur (Encoder) : Il prend les données d'entrée et les compresse en une représentation de plus petite dimension appelée code latent ou bottleneck.

☞ C'est souvent **une série de couches denses** ou convolutives avec des fonctions d'activation (comme ReLU).

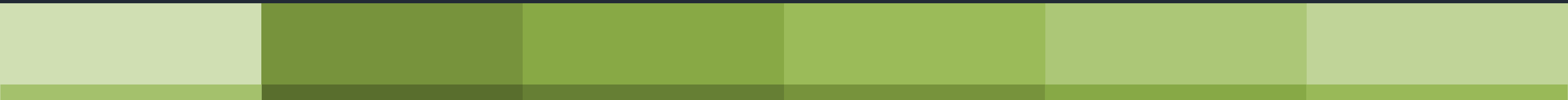
2.Bottleneck (ou espace latent) : C'est **le cœur de l'autoencodeur**. Il contient la version compressée de l'information.

☞ Plus cet espace est petit, plus le réseau est forcé de garder uniquement les caractéristiques les plus importantes.

3.Décodeur (Decoder) : Il prend la représentation comprimée et tente de reconstruire l'entrée originale.

☞ L'objectif est que la sortie soit aussi proche que possible de l'entrée initiale.

Comprendre les Autoencodeurs : Les notions essentielles



Les notions essentielles

1. Apprentissage non supervisé

Les autoencodeurs font partie des techniques **d'apprentissage non supervisé**.

=> Leur but principal est **d'apprendre à reproduire les données d'entrée** en les reconstruisant aussi fidèlement que possible.

=> Pendant l'entraînement, le réseau apprend à minimiser l'écart entre l'entrée et la sortie.

2. Compression avec perte d'information

Les autoencodeurs réalisent une compression dite "**avec perte**".

=> Cela signifie qu'ils réduisent la taille des données pour **en extraire l'essentiel**, mais certaines informations peuvent être perdues au passage.

=> C'est utile lorsque l'on veut simplifier les données ou les stocker plus efficacement.

3. Spécialisation sur les données d'entraînement

Les autoencodeurs apprennent les structures spécifiques aux données sur lesquelles ils sont entraînés.

=> Ils deviennent donc très efficaces pour ces données, mais **ne s'adaptent pas facilement à des données très différentes**.

Les notions essentielles

Un autoencodeur peut-il être utilisé dans un cadre d'apprentissage supervisé, et si oui, dans quelles conditions et dans quel but ?

En général, un autoencodeur est un modèle d'apprentissage non supervisé. Cela signifie qu'il n'utilise pas d'étiquettes (labels) pour apprendre : il s'entraîne simplement à reconstruire ses propres entrées.

Cependant, il peut être utilisé dans des contextes supervisés de façon indirecte, par exemple :

1. En prétraitement pour un apprentissage supervisé :

L'autoencodeur apprend à compresser les données dans un espace latent, et cette représentation peut ensuite être utilisée comme entrée d'un modèle supervisé (par exemple, un classificateur).

👉 Ex : entraînement d'un autoencodeur sur des images, puis utilisation de la couche latente comme vecteurs d'entrée pour une classification.

Les notions essentielles

Un **autoencodeur** peut-il être utilisé dans un cadre d'apprentissage supervisé, et si oui, dans quelles conditions et dans quel but ?

2. En mode semi-supervisé :

On peut combiner l'objectif de reconstruction avec une tâche supervisée (comme la classification), **en ajoutant une couche de sortie avec des étiquettes**. Le modèle est alors entraîné à la fois à reconstruire les données et à prédire les classes.

Remarque

Pour Bien Comprendre Je veux conseille de voir d'abord la structure d'un autoncoder simple puis et aussi le code dans le notebook : **Auto-encoder-avec-deux-sortie.ipynb**

```
[7]:  
  
# Definition de l'encodeur  
inputs = Input(shape=(28*28,)) # Image d'entree de taille 28*28  
x = Dense(128, activation='relu')(inputs)  
x = Dense(64, activation='relu')(x)  
latent = Dense(latent_dim, activation='relu')(x) # Espace latent  
  
[8]:  
  
# Definition du decodeur  
x = Dense(64, activation='relu')(latent)  
x = Dense(128, activation='relu')(x)  
decoded = Dense(28*28, activation='sigmoid')(x) # Image reconstruite (sortie)  
  
[9]:  
  
# Couche de classification  
classification = Dense(10, activation='softmax')(latent) # Prediction de la classe  
  
[10]:  
  
# Modele final  
autoencoder = Model(inputs, [decoded, classification])
```


Étapes de Conception



Étapes de Conception

Comment construire un **Autoencodeur** ?

1. **Choix de la taille d'entrée** (en fonction de vos données).
2. **Définir l'architecture** :
 - a. Nombre de couches dans l'encodeur/décodeur.
 - b. Taille de l'espace latent.
3. **Choix des fonctions d'activation** (ReLU, sigmoid, tanh, etc.).
4. **Fonction de perte (loss function)** : Généralement, l'erreur quadratique moyenne (MSE) est utilisée pour mesurer la différence entre l'entrée et la sortie.
5. **Optimisation** : Utilisation d'un algorithme comme Adam ou SGD pour minimiser la perte.
6. **Entraînement** : Le réseau apprend à reconstruire les entrées à partir des représentations compressées.

Exemple

Auto-Encoder simple avec MNIST

1. **Jeu de données : MNIST (images 28x28 pixels)**
2. **Définir l'architecture et Choix des fonctions d'activation :**
 - a. encoder : [Couche d'entrée : 784 neurones], [Couche dense : 128 neurones, ReLU] , [Couche dense : 64 neurones, ReLU]
 - b. dencoder : [Couche dense : 128 neurones, ReLU], [Couche de sortie : 784 neurones, Sigmoid]
 - c. Taille de l'espace latent : 64 neurones
3. **Fonction de perte (loss function) :** binary_crossentropy
4. **Optimisation :** Adam pour minimiser la perte.
5. **Entraînement :** Le réseau apprend à reconstruire les entrées à partir des représentations compressées.
6. **Evaluation :** visualiser les images originales vs reconstruites

Intérêt de Cet Exemple

Pourquoi procéder à la compression des données si l'objectif final est simplement de retrouver l'entrée initiale ?

La compression des données via un autoencodeur ne vise pas uniquement à retrouver l'entrée initiale, mais à **apprendre une représentation compacte et significative des données**. Cette représentation — appelée espace latent — permet de **capturer les caractéristiques essentielles** tout en éliminant les redondances et le bruit. Ainsi, même si le modèle reconstruit l'entrée, l'intérêt principal réside dans **l'exploitation de cette représentation compressée** pour d'autres tâches utiles comme :

- la classification (en utilisant l'espace latent comme vecteur de caractéristiques), ex : TP
- la réduction de bruit (reconstruction d'une version propre d'une entrée bruitée),
- la détection d'anomalies (en identifiant les entrées mal reconstruites),
- ou encore la génération de nouvelles données (dans les autoencodeurs variationnels).

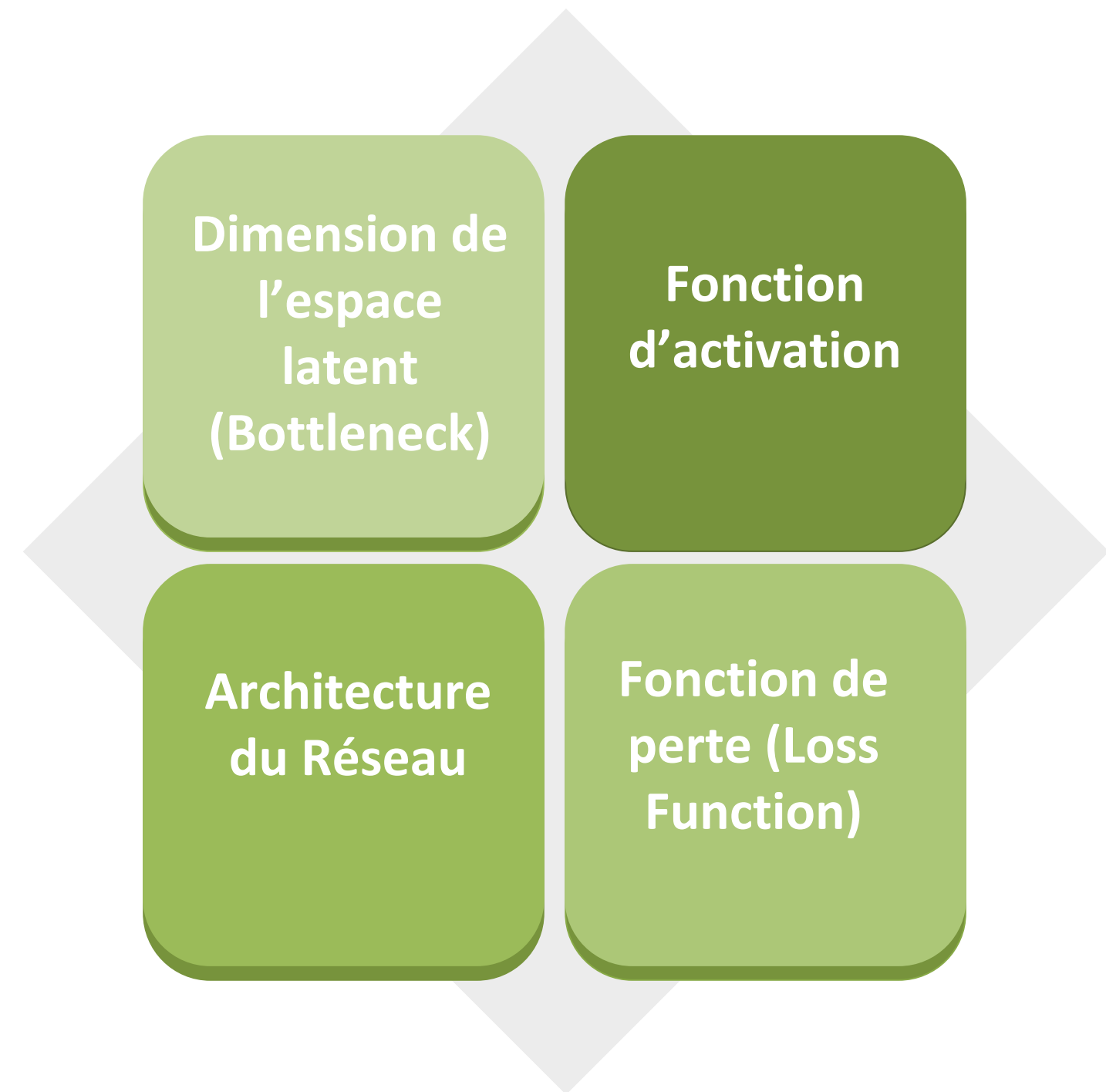
Ajustement des Hyperparamètres d'un Autoencodeur



Ajustement des Hyperparamètres d'un Autoencodeur

Un **autoencodeur performant** dépend fortement du **choix** adéquat de **ses hyperparamètres**.

L'ajustement de ces paramètres est essentiel pour garantir que le modèle apprenne efficacement et produise des reconstructions de qualité. Voici les principaux hyperparamètres à considérer et comment les ajuster :



Ajustement des Hyperparamètres d'un Autoencodeur

1. Dimension de l'espace latent (Bottleneck) : La taille de l'espace latent détermine combien d'informations le modèle peut conserver après avoir compressé les données d'entrée.

> **Un espace latent trop petit** pourrait entraîner une **perte excessive d'informations**,
> tandis qu'un **espace trop grand** pourrait rendre le modèle trop flexible et **risquer de sur-apprendre** les données.

👉 **Comment ajuster :** Essayez différentes tailles d'espace latent, comme 32, 64, ou 128, et observez l'impact sur la qualité de la reconstruction.

Ajustement des Hyperparamètres d'un Autoencodeur

2. Architecture du réseau (Nombre de couches et neurones) :

- > Le nombre de **couches** et de **neurones** dans chaque couche joue un rôle clé dans l'efficacité de l'apprentissage.
- > **trop peu de couches** ou de neurones peuvent rendre **le modèle trop simple**, incapable d'apprendre des représentations utiles.
- > **Trop de couches** ou de neurones peuvent entraîner **un modèle trop complexe**, susceptible de sur-apprendre les données .

👉 **Comment ajuster** : Testez différentes configurations d'architecture, par exemple un encodeur avec 3 couches vs 5 couches, et ajustez le nombre de neurones (par exemple, 64 ou 128) pour trouver l'équilibre entre complexité et performance.

Ajustement des Hyperparamètres d'un Autoencodeur

3. Fonction d'activation : Les fonctions d'activation ajoutent de la non-linéarité et permettent au modèle de **capturer des relations complexes entre les données**. Les fonctions comme ReLU (Rectified Linear Unit) sont souvent utilisées dans les autoencodeurs, mais selon le problème, des fonctions comme sigmoid ou tanh peuvent également être adaptées.

👉 **Comment ajuster :** Testez différentes fonctions d'activation : par exemple, ReLU dans l'encodeur et sigmoid dans le décodeur pour des sorties comprises entre 0 et 1.

Recommandations pratiques pour l'autoencodeur :

=> **Utilisez ReLU** dans l'encodeur (partie qui compresse les données) pour une bonne performance et pour capturer des représentations efficaces des données.

=> **Sigmoid** Parfait pour des problèmes **où la sortie doit être probabilistique**, comme la modélisation des événements de type binaire (par exemple, la présence ou l'absence d'un objet dans une image)

Ajustement des Hyperparamètres d'un Autoencodeur

4. Fonction de perte (Loss Function) : La fonction de perte mesure la différence entre l'entrée et la sortie du modèle. La fonction la plus couramment utilisée dans les autoencodeurs est **l'erreur quadratique moyenne** (MSE). Cependant, **si les données sont binaires**, une fonction comme **binary_crossentropy** pourrait être plus appropriée.

👉 **Comment ajuster :** Utilisez MSE pour des données continues. Pour des données binaires (comme des images binaires), optez pour `binary_crossentropy`.

Types of Autoencoders



Types of Autoencoders

Denoising Autoencoder

Il est entraîné à travailler avec des entrées corrompues ou bruitées, et apprend à éliminer le bruit pour reconstruire les données originales et propres.

Variational Autoencoder

Il fait des hypothèses sur la distribution de probabilité des données et cherche à apprendre une approximation plus fidèle de cette distribution. Il est souvent utilisé pour la génération de données.

Sparse Autoencoder

Il contient plus de neurones cachés que d'entrées, mais seulement quelques-uns sont autorisés à s'activer simultanément. Cela permet d'apprendre des représentations plus efficaces et interprétables.

Convolutional Autoencoder

L'encodeur utilise des couches convolutionnelles pour extraire les caractéristiques d'une image, et le décodeur applique des déconvolutions pour reconstruire l'image.

Denoising Autoencoder

Un autoencodeur de débruitage (denoising autoencoder) : il s'entraîne à reconstruire l'original propre à partir d'une version corrompue $\tilde{x}$. Cette corruption peut être un bruit gaussien, un masquage de pixels, etc.

> **Le bruit gaussien** (ou bruit blanc gaussien) est un **type de perturbation aléatoire** dont les valeurs suivent une loi normale (distribution gaussienne) de densité de probabilité définie par :

$$p(z) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(z - \mu)^2}{2\sigma^2}\right)$$

Principe de fonctionnement :

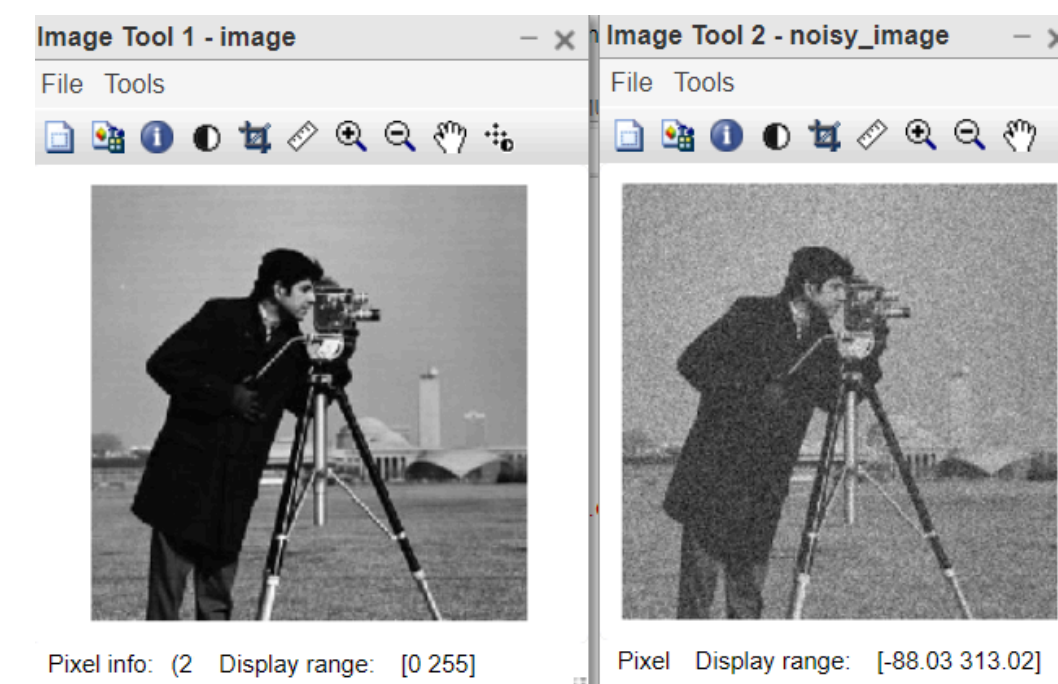
1. Ajout de bruit : On perturbe délibérément l'entrée :

$$\tilde{x} = x + \text{bruit}$$

Par exemple, pour une image on peut ajouter du bruit gaussien, ou remplacer aléatoirement un certain pourcentage de pixels par 0 (masquage).

2. Encodage : Le réseau encode la version bruitée $\tilde{x}$ en un vecteur latent $h = f_{\theta}(\tilde{x})$

3. Décodage : À partir de h , le décodeur reconstruit une estimation



Sparse Autoencoder

Un Autoencodeur Creux (Sparse Autoencoder): impose que seuls **quelques neurones de la couche latente s'activent pour chaque entrée**, produisant un code clairsemé (sparse) et plus interprétable.

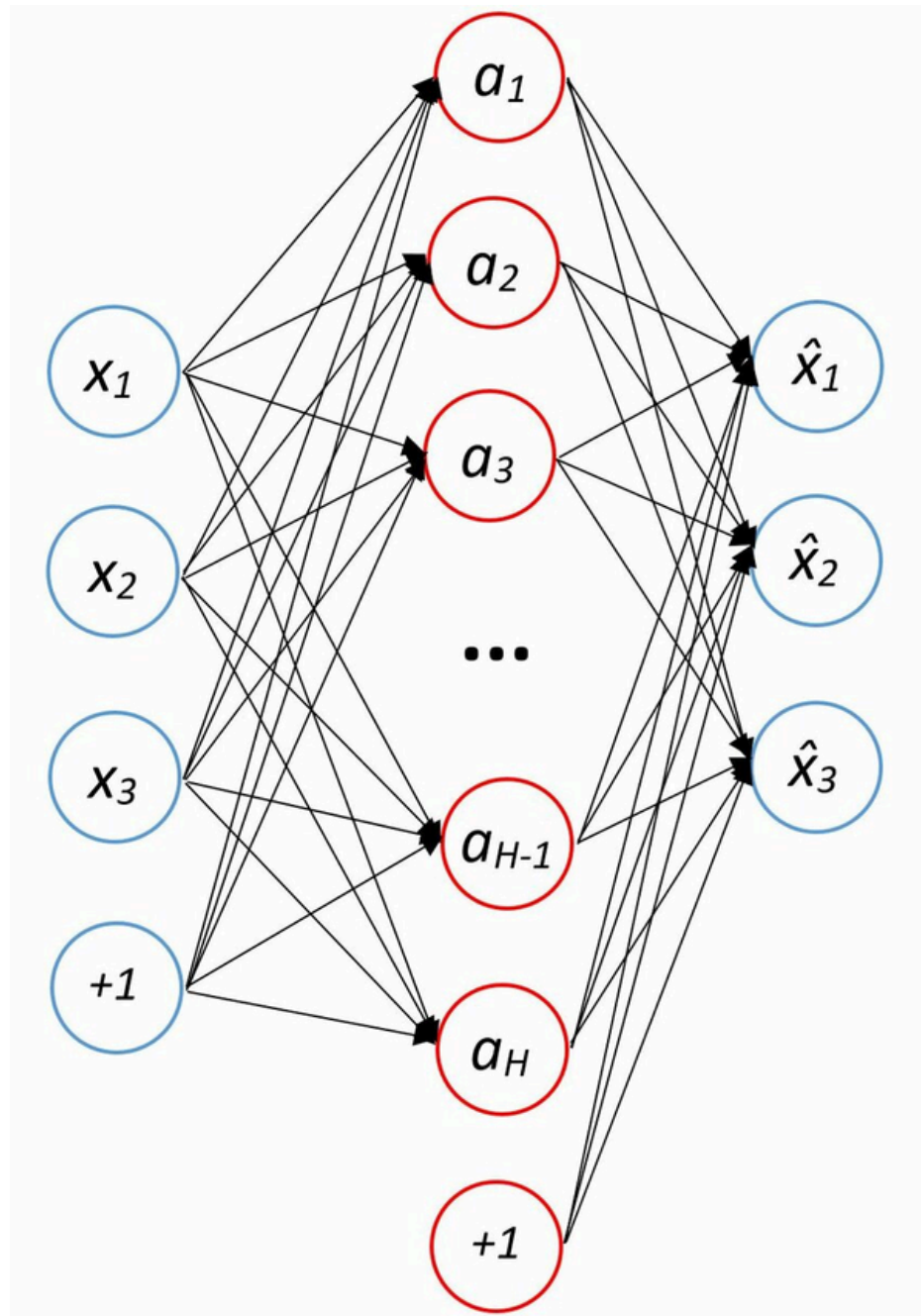
Principe de la sparsité :

- On définit un taux de sparsité (ex. 0,05).
- Pour chaque neurone , on mesure sa moyenne d'activation sur un lot d'exemples.
- On pénalise l'écart par rapport à via la divergence de Kullback–Leibler :

$$\Omega = \sum_j [\rho \log(\rho/\hat{\rho}_j) + (1 - \rho) \log((1 - \rho)/(1 - \hat{\rho}_j))]$$

- Fonction de coût :

$$\mathcal{L}_{\text{sparse}} = \mathcal{L}_{\text{mse}} + \beta \Omega$$



Convolutional Autoencoder

Un **Convolutional Autoencoder (CAE)** : est un type d'autoencodeur qui utilise **des couches convolutionnelles** (les mêmes que dans les CNN, ou réseaux convolutifs) au lieu de simples couches entièrement connectées (dense).

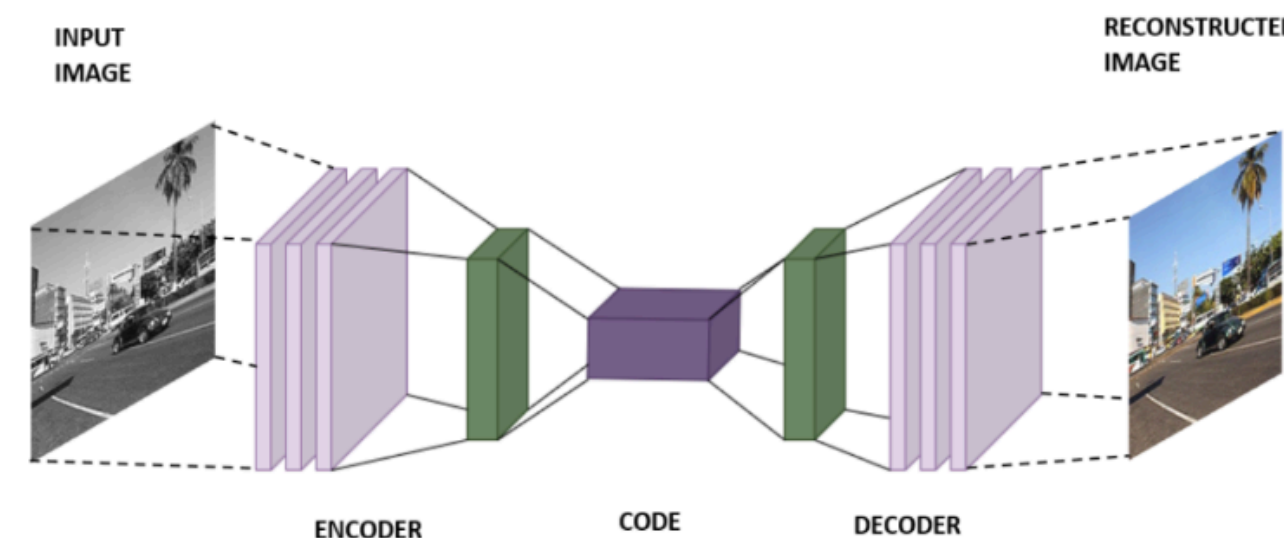
Il est particulièrement **adapté au traitement d'images**.

Principe de Fonctionnement :

- Des couches convolutionnelles dans l'encodeur pour **extraire les caractéristiques importantes**.
- Des **couches deconvolutionnelles** (ou upsampling) dans le décodeur pour reconstruire l'image.

Par exemple on peut l'utiliser dans la coloration des images .

- on commence par **prendre une image** en niveaux de gris
- L'encodeur utilise des **couches convolutives** pour extraire les **caractéristiques importantes** de l'image (bords, textures, etc.).
- L'encodeur produit une représentation latente comprimée qui contient **les informations essentielles** permettant de reconstruire l'image.
- Le décodeur prend la représentation latente obtenue de l'encodeur et tente de reconstruire les canaux de l'image colorisée en utilisant des **couches de convolutions transposées**



Variational Autoencoder (VAE)

Un VAE : est un type d'**autoencodeur probabiliste**. Il ne code pas directement une entrée vers une représentation fixe, mais vers une distribution de probabilité.

Autrement dit, au lieu de produire un seul "code latent", il **produit des paramètres d'une distribution** (souvent une gaussienne) : **la moyenne (μ) et l'écart-type (σ)**.

Principe de fonctionnement :

1.Encodeur :

- Prend une image (ou autre donnée) en entrée
- Produit deux vecteurs : μ (la moyenne) et σ (l'écart-type)
- Représente la distribution latente de cette entrée

2.Sampling (échantillonnage) :

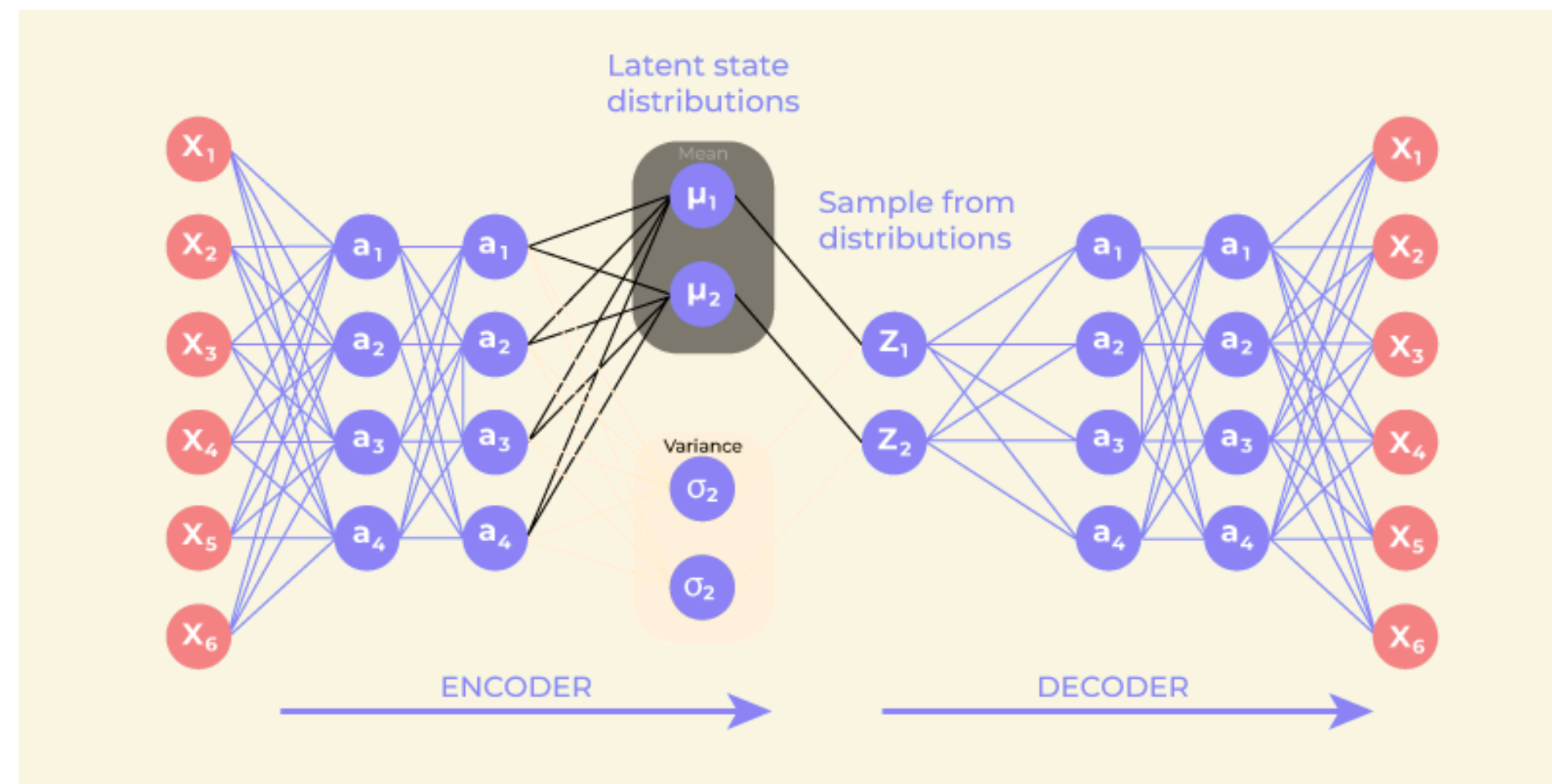
- Tire un vecteur latent z depuis une gaussienne :
- $z = \mu + \sigma \cdot \epsilon$ où $\epsilon \sim N(0,1)$
- Cette astuce s'appelle le "reparameterization trick" (pour que le réseau reste entraînable avec backpropagation)

3.Décodeur :Reconstruit l'image à partir de ce vecteur latent z

Variational Autoencoder (VAE)

Un **VAE** : est un type d'**autoencodeur probabiliste**. Il ne code pas directement une entrée vers une représentation fixe, mais vers une distribution de probabilité.

Autrement dit, au lieu de produire un seul "code latent", il **produit des paramètres d'une distribution** (souvent une gaussienne) : **la moyenne (μ) et l'écart-type (σ)**.



Variational Autoencoder (VAE)

Fonction de perte dans un VAE :

Contrairement à un autoencodeur classique (où on mesure juste la reconstruction), ici on a deux composantes :

1. **Erreur de reconstruction** (comme un autoencodeur classique)
2. **KL divergence** : mesure à quel point la distribution $N(\mu, \sigma)$ s'éloigne d'une gaussienne standard $N(0,1)$

La perte totale devient :

$$\text{Loss} = \text{Reconstruction Loss} + \text{KL Divergence}$$

Avec KL Divergence =

$$\mathcal{L}_{\text{KL}} = -\frac{1}{2} \sum_{i=1}^d (1 + \log(\sigma_i^2) - \mu_i^2 - \sigma_i^2)$$

où :

- d : dimension du vecteur latent z
- μ_i : la moyenne estimée pour la i -ème dimension de z
- σ_i^2 : la variance (au carré) estimée pour la i -ème dimension de z

Real-World Applications



Applications des auto-encodeurs

Réduction de bruit sur les images

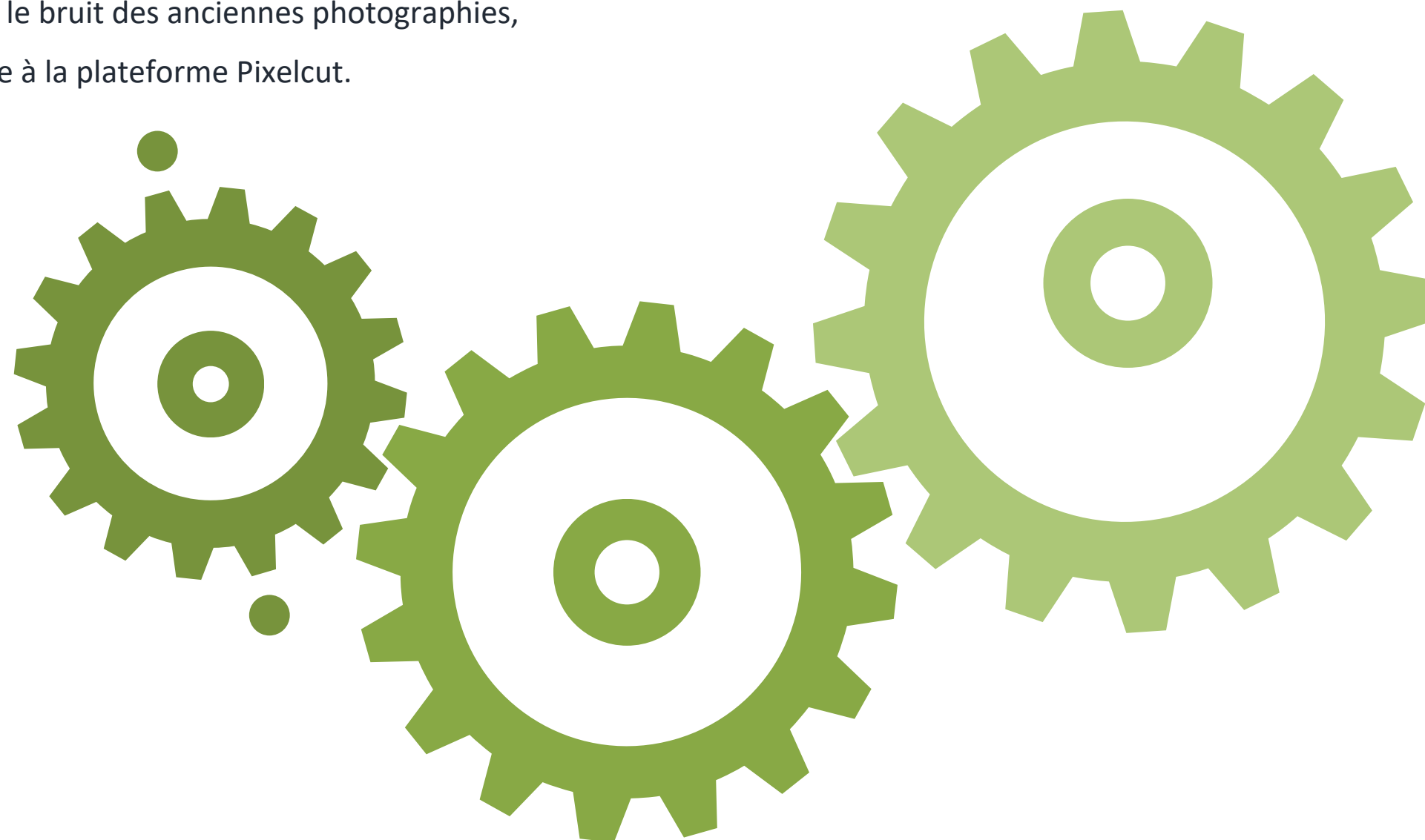
Élimine le bruit des anciennes photographies, similaire à la plateforme Pixelcut.

Détection des anomalies

Entraînez un autoencodeur sur des données normales, puis il apprend à reconstruire correctement les entrées normales.

Génération de données

Art généré par IA et vidéos deepfake, comme sur DeepAI et OpenArt.



Implementation de Auto-encodeur



**THANK YOU FOR
YOUR ATTENTION !!!**

